

Machine Learning and Data Mining (Module 2)

Matteo Francia
DISI — University of Bologna
m.francia@unibo.it

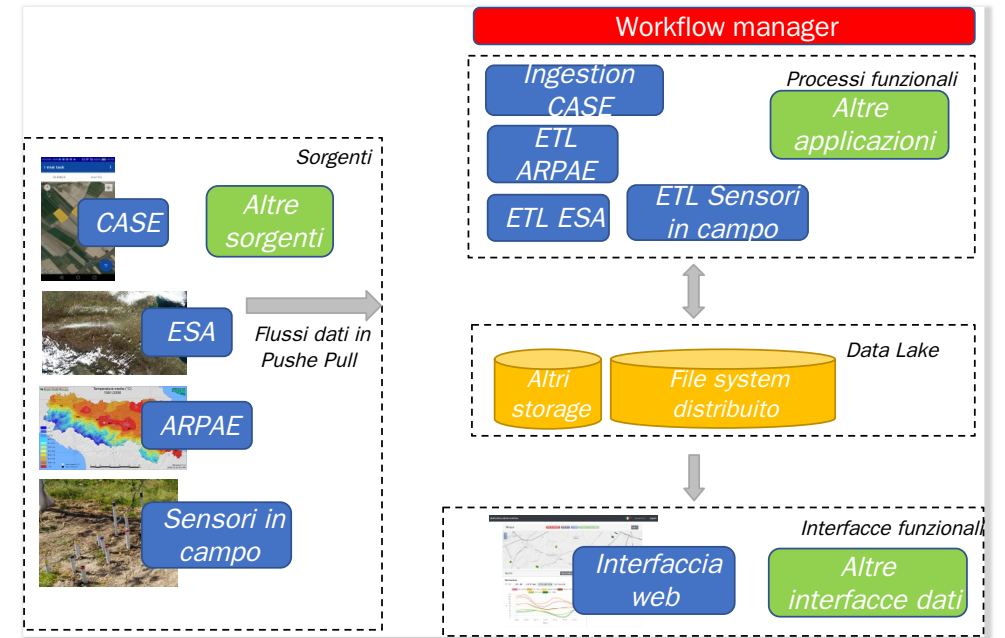
A telemetry system for precision agriculture

The impacts of climate change are so rapid that what happened last year is no longer true today.

- We need to act quickly and make decisions based on evidence, not just history.



F1 control room



The "Agro.Big.Data.Science" platform

ABDS: A modular platform for collecting, analyzing, and visualizing heterogeneous data from the field and the post-harvest phase.

Cimice.Net

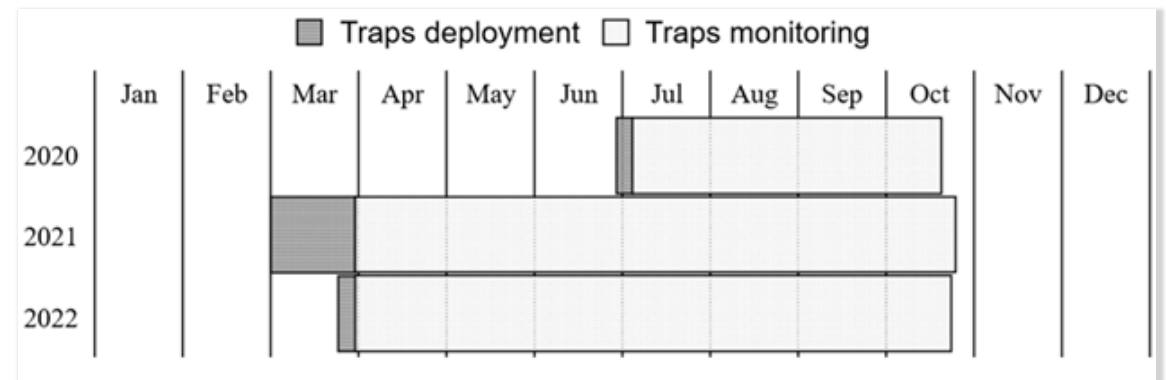
The brown marmorated stink bug (Halyomorpha halys) is an insect pest species causing economic damage to several agricultural commodities

- We want to build a data-driven approach to support the application of Integrated Pest Management strategies
- We want to monitor the spread of H. Halys
- And learn the most important environmental factors

Three years-long project



Brown marmorated stink bug



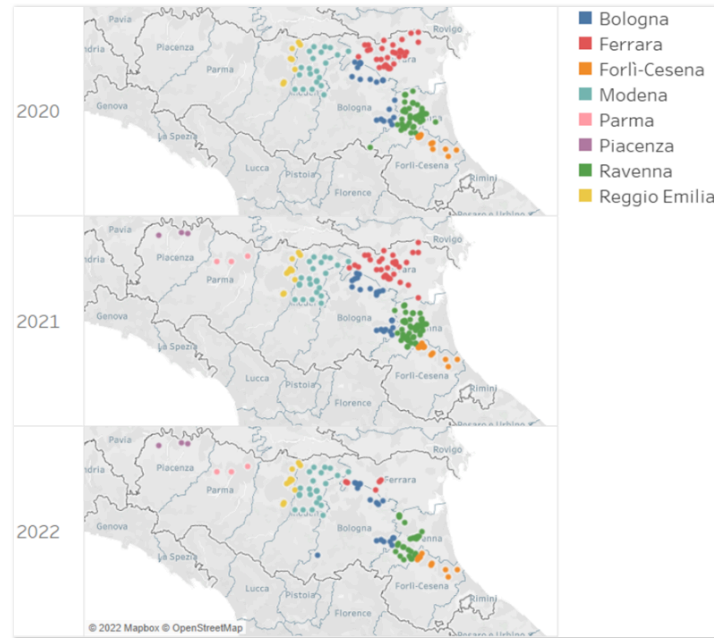
Cimice.Net

Goal: help farmers protect crops

- A network of monitoring traps has been deployed in Emilia-Romagna
- Monitoring 145, 168, and 101 farms in 2020-2022 (also in 2023 and 2024)



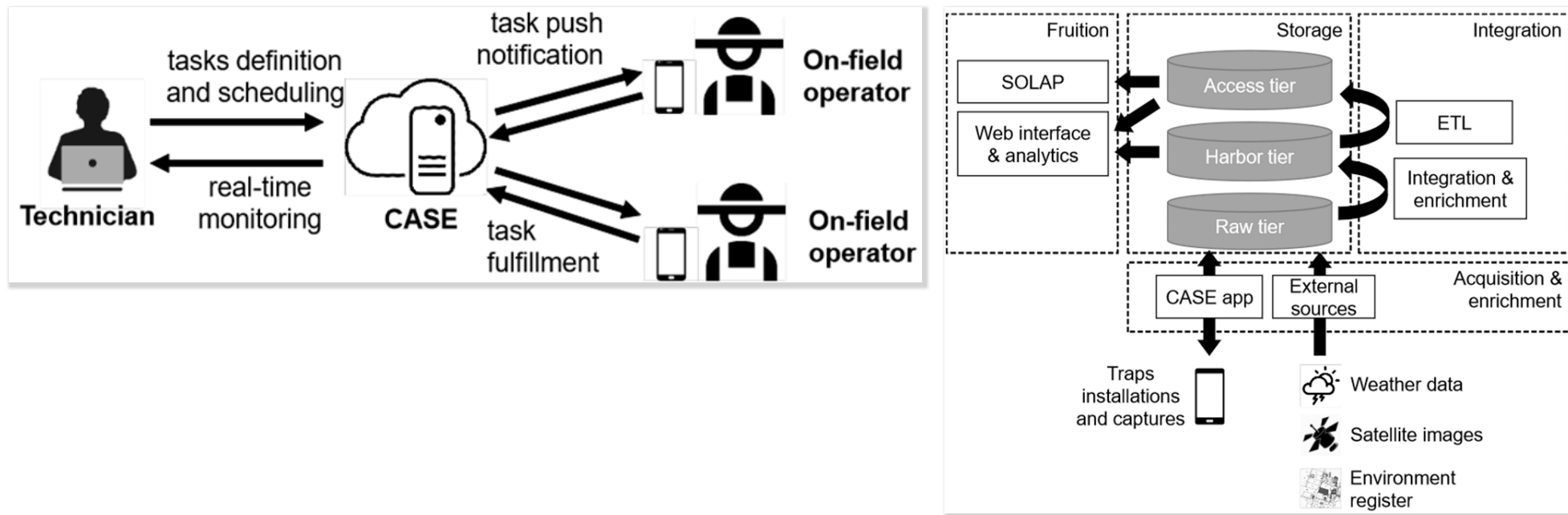
Trap

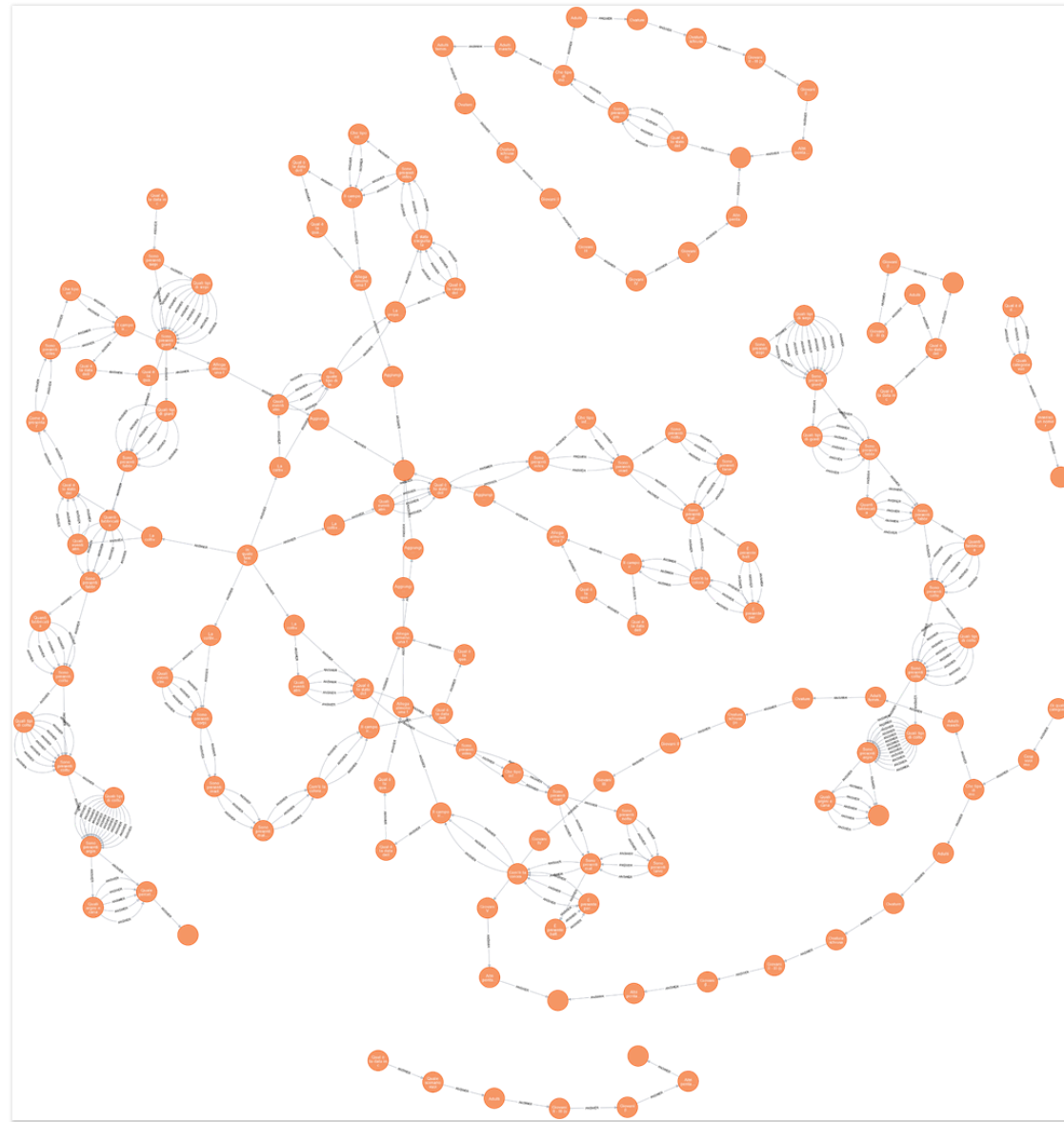


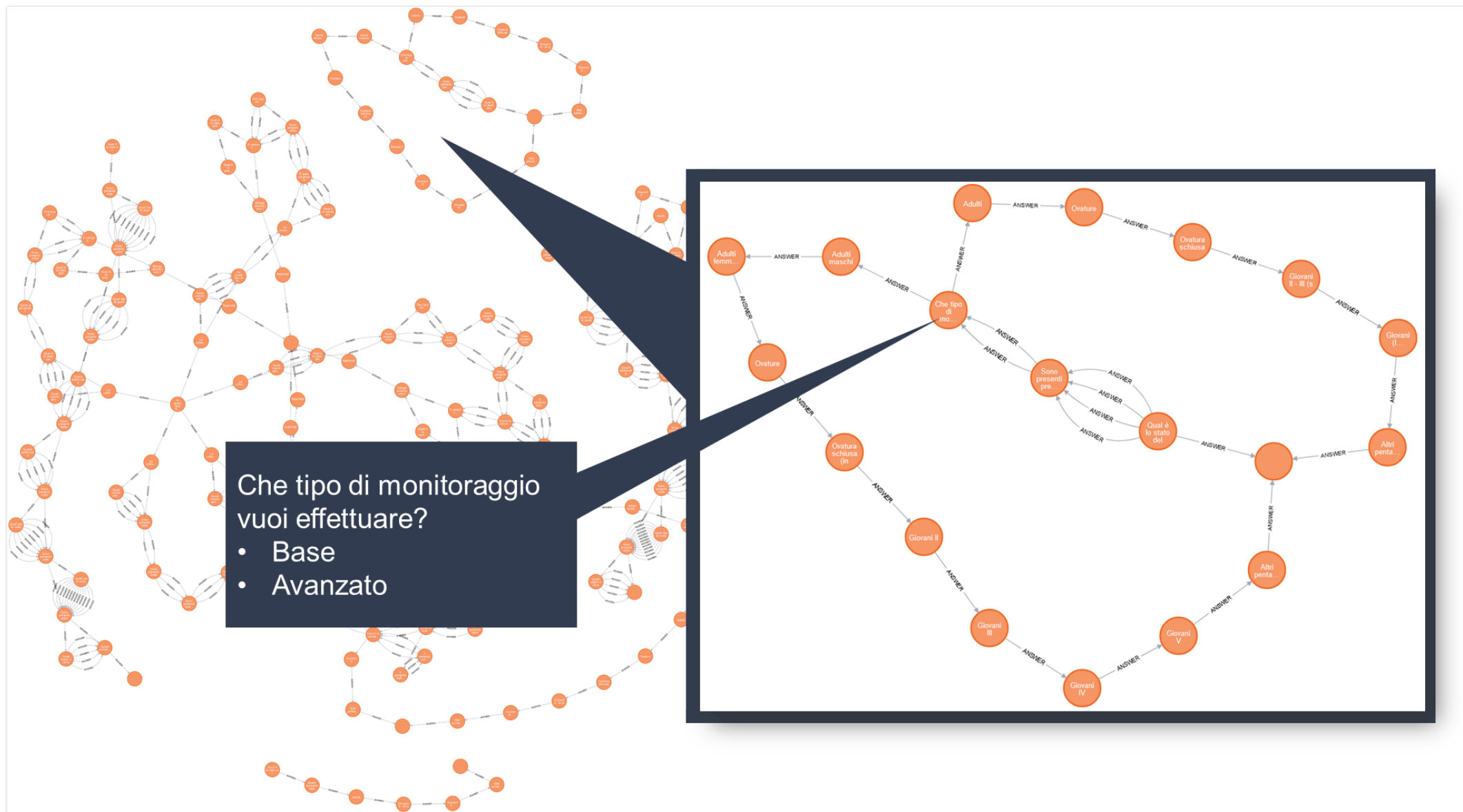
Collaborative Agro SEnsing

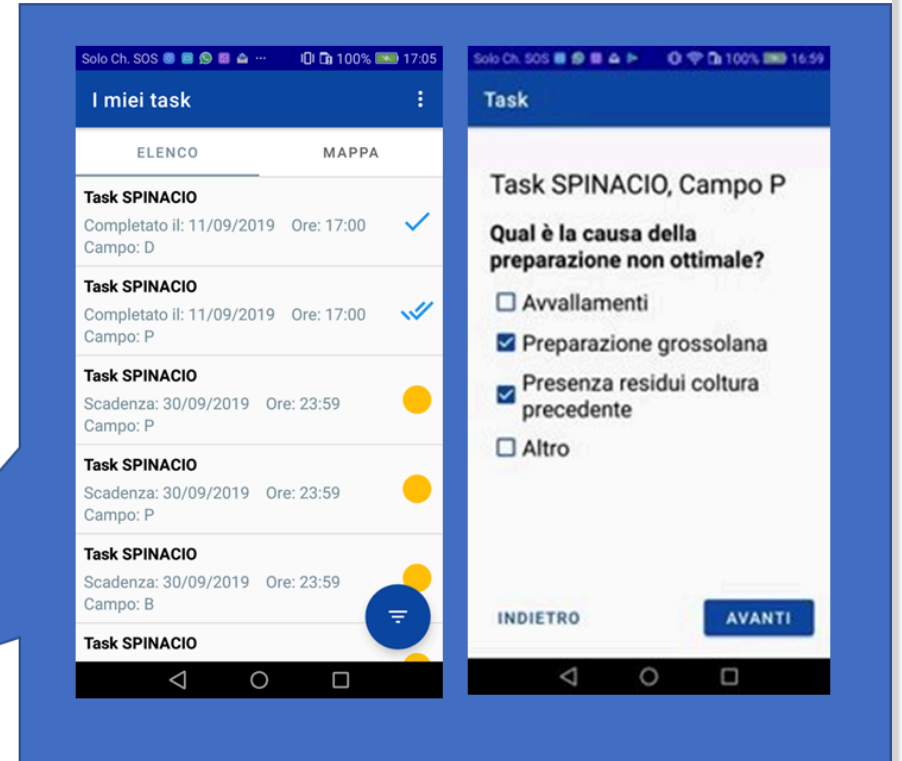
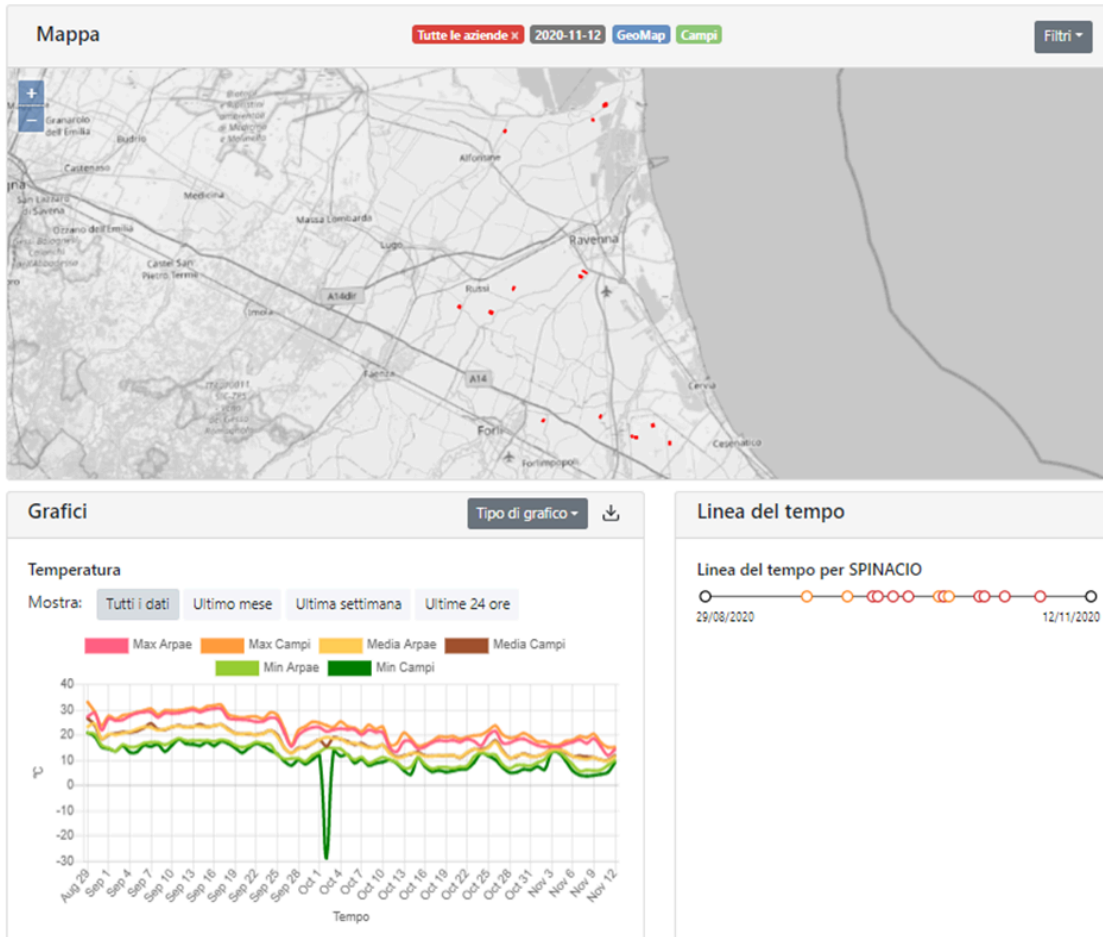
The acquisition of data concerning the installation and monitoring of traps have been aided by *CASE* (*Collaborative Agro SEnsing*)

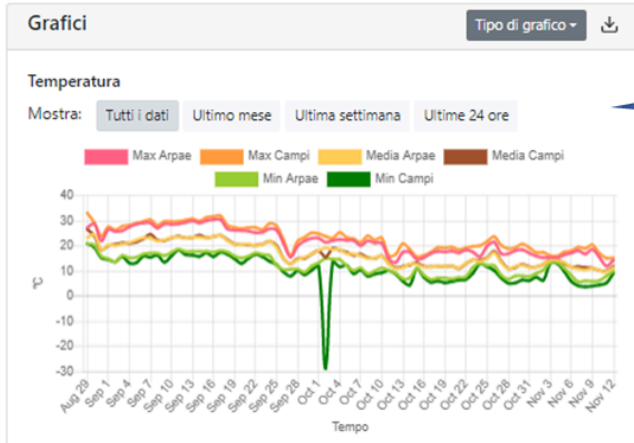
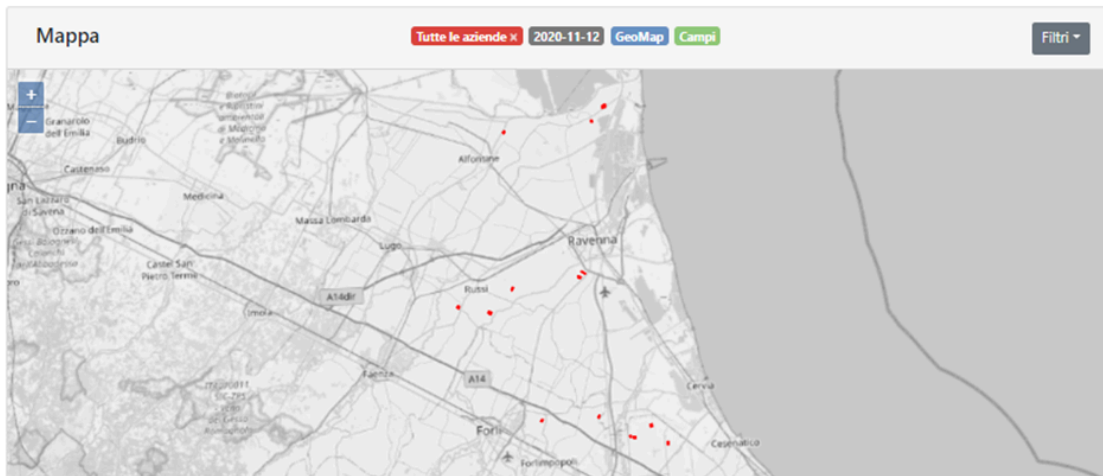
- Dynamic questionnaire application for on-field data crowdsourcing in the agricultural domain
- Facilitate and standardize the communications between on-field operators with first-hand visuals of a given field/orchard and the technicians needing a 360-degree view of all fields





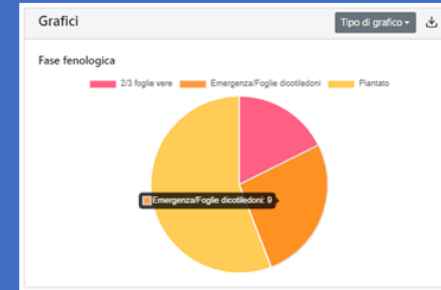
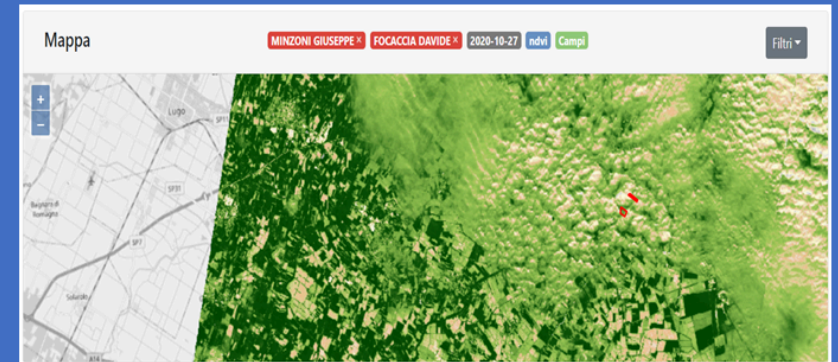




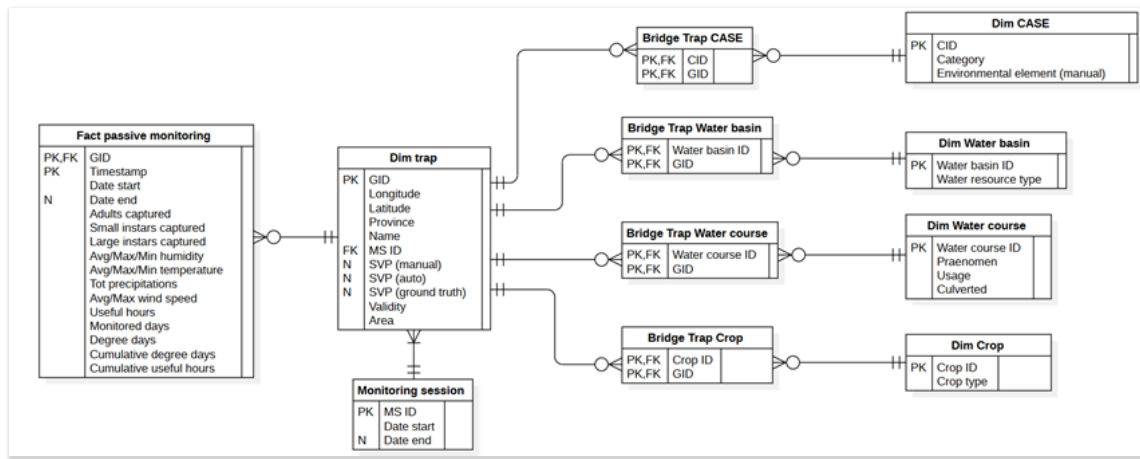


Linea del tempo

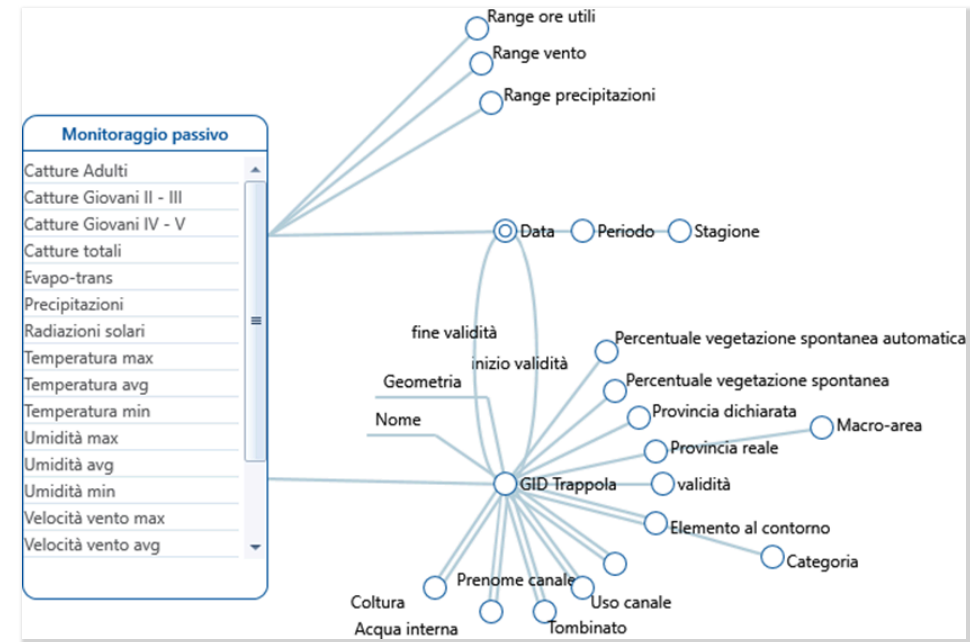
Linea del tempo
29/08/2020



Fase fenologica
Stima della
quantità raccolta



Database



Multidimensional cube (DFM)

Live!

The `Stinkbug` dataset

Loading the data

▼ Code

```
1 import pandas as pd
2 df = pd.read_csv("./datasets/cimice/captures-raw.csv", sep=',')
3 df
```

	GID	TIMESTAMP	CROP_ID	ADULTS	SMALL_INSTARS	LARGE_INSTARS	TEMPERATURE_AVG	TEMPERATURE_MAX	TEMPERATURE_MIN	HUMIDITY_AVG	HUMIDITY_MAX	HUMIDITY_MIN	PRECIPITATIONS	WI
0	149	2020-04-27	41	0.0	0.0	0.0	16.966669	22.683333	10.550002	63.500000	84.833333	41.500000	2.199999	1.5
1	144	2020-04-27	31	0.0	0.0	0.0	15.785717	22.885719	8.042862	64.571429	90.142857	39.000000	4.000001	1.6
2	215	2020-04-27	47	5.0	0.0	0.0	17.037496	22.787504	11.149999	53.000000	72.375000	36.250000	2.599999	2.6
3	222	2020-04-27	19	1.0	0.0	0.0	16.942861	23.457143	9.414279	59.857143	90.571429	38.142857	3.299999	2.8
4	217	2020-04-27	19	9.0	0.0	0.0	17.028571	22.885710	10.371432	58.428571	84.428571	39.571429	5.000000	2.6
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
10944	810	2022-10-17	19	10.0	0.0	0.0	17.687498	24.512502	12.149995	76.125000	NaN	NaN	0.200000	Na
10945	845	2022-10-17	39	26.0	0.0	0.0	18.025003	23.975000	13.274995	76.500000	NaN	NaN	0.000000	Na
10946	888	2022-10-17	11	1.0	0.0	0.0	16.900003	23.875002	10.900003	77.500000	NaN	NaN	0.400000	Na
10947	885	2022-10-17	36	16.0	0.0	0.0	17.599991	23.614282	12.300003	76.857143	NaN	NaN	0.299999	Na
10948	815	2022-10-17	41	3.0	0.0	0.0	17.175001	23.887502	11.062498	81.750000	NaN	NaN	0.600001	Na

10949 rows × 16 columns

... and then?

How many of you apply **GenAI** (e.g., LLMs such as ChatGPT) to data science?

(Dis)Agree? ([Welsh 2023](#))

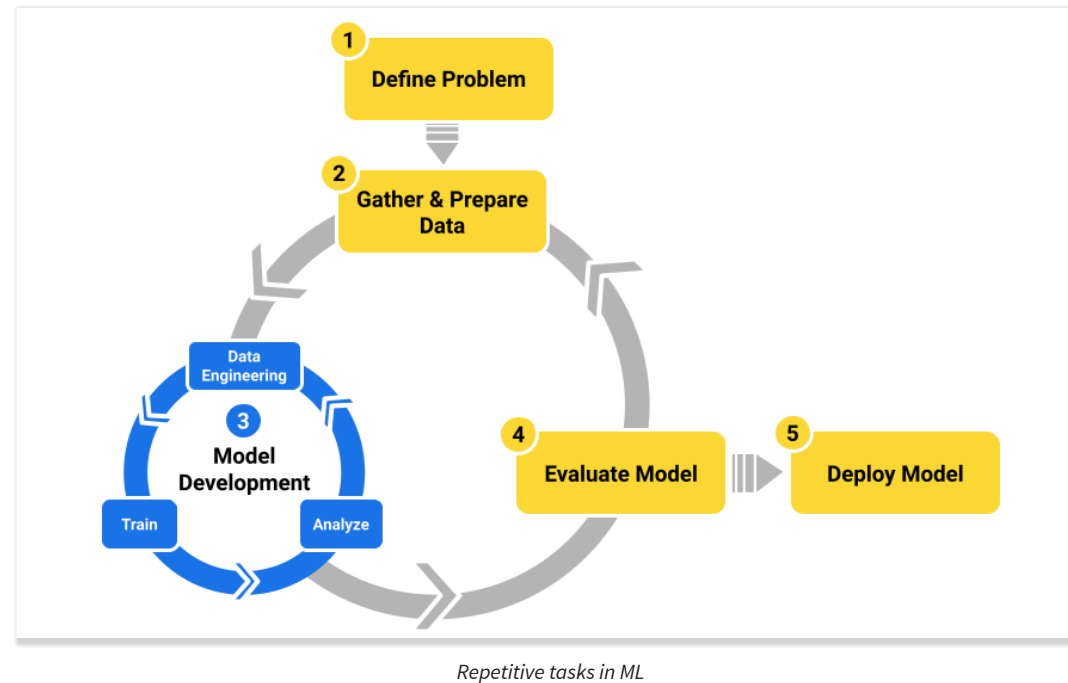
“Programming will be obsolete. I believe the conventional idea of ‘writing a program’ is headed for extinction, and indeed, for all but very specialized applications, most software, as we know it, will be replaced by AI systems that are trained rather than programmed. In situations where one needs a ‘simple’ program (after all, not everything should require a model of hundreds of billions of parameters running on a cluster of GPUs), those programs will, themselves, be generated by an AI rather than coded by hand.”

(Waldo and Boussard 2025)

Despite their powerful capabilities, however, generative pre-trained transformers (GPTs) have the tendency to “hallucinate” responses. A hallucination occurs when an LLM-based GPT generates a response that is seemingly realistic yet is nonfactual, nonsensical, or inconsistent with the given prompt.

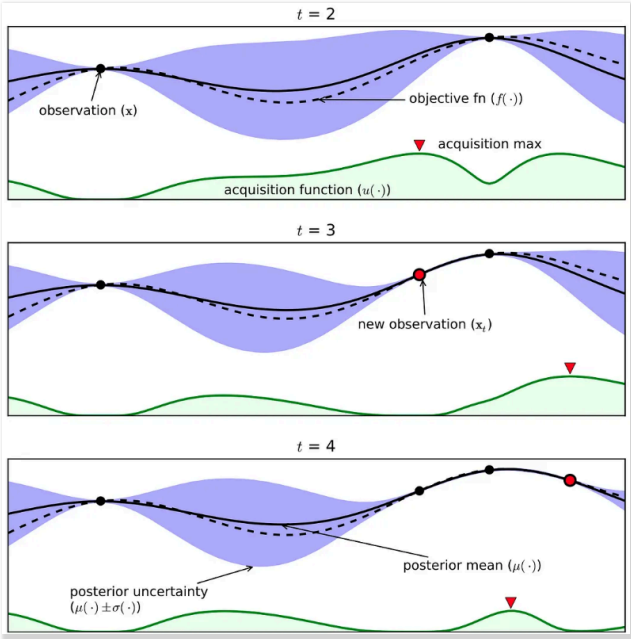
How many of you use **AutoML** for training Machine Learning pipelines?

AutoML in brief



Check the crash course from [Google](#)

AutoML in brief



Example of Bayesian Optimization

(Bie et al. 2022)

“**AutoML** falls squarely into the first form of automation, mechanization [...] as it can be seen as yet another level of abstraction over a series of automation stages.

1. First, there is the well-known use of *programming for automation*.
2. Second, *machine learning automatically generates hypotheses and predictive models*, which typically take the form of algorithms (for example, in the case of a decision tree or a neural network); therefore, machine learning methods can be seen as *meta-algorithms that automate programming tasks*, and hence “automate automation.”
3. And third, *automated machine learning makes use of algorithms that select and configure machine learning algorithms*—that is, of *meta-meta-algorithms that can be understood as automating the automation of automation*.”

(Bie et al. 2022)

“The impressive performance levels reached by AutoML systems are evident in the results from recent competitions.

Notably, Auto-sklearn significantly outperformed human experts in the human track of the 2015/2016 ChaLearn AutoML Challenge.”

Do we still need data scientists?

GenAI and AutoML

This **code was written by ChatGPT 4o** and uses **AutoML to train several models** given a specific budget

▼ Code

```
1 from sklearn.metrics import root_mean_squared_error
2 from sklearn.model_selection import train_test_split
3 from flaml import AutoML
4
5 X = df.drop(columns=['ADULTS'])
6 y = df['ADULTS']
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
8
9 automl = AutoML()
10 automl_settings = {
11     "time_budget": 30, # Time budget in seconds
12     "max_iter": 30, # Max number of iterations
13     "metric": 'rmse', # Evaluation metric
14     "task": 'regression', # Task type
15 }
16 automl.fit(X_train, y_train, **automl_settings)
17 y_pred = automl.predict(X_test)
18 rmse = root_mean_squared_error(y_test, y_pred)
```

```
[flaml.automl.logger: 04-08 20:44:01] {1728} INFO - task = regression
[flaml.automl.logger: 04-08 20:44:01] {1739} INFO - Evaluation method: holdout
[flaml.automl.logger: 04-08 20:44:00] {1838} INFO - Minimizing error metric: rmse
[flaml.automl.logger: 04-08 20:44:00] {1955} INFO - List of ML learners in AutoML Run: ['lgbm', 'rf', 'xgboost', 'extra_tree', 'xgb_limitdepth', 'sgd']
[flaml.automl.logger: 04-08 20:44:00] {2258} INFO - iteration 0, current learner lgbm
[flaml.automl.logger: 04-08 20:44:00] {2393} INFO - Estimated sufficient time budget=0s. Estimated necessary time budget=0s.
[flaml.automl.logger: 04-08 20:44:00] {2442} INFO - at -0.5s, estimator lgbm's best error=12.0178, best estimator lgbm's best error=12.0178
[flaml.automl.logger: 04-08 20:44:00] {2258} INFO - iteration 1, current learner rf
[flaml.automl.logger: 04-08 20:44:00] {2442} INFO - at -0.4s, estimator rf's best error=7.9172, best estimator rf's best error=7.9172
[flaml.automl.logger: 04-08 20:44:00] {2258} INFO - iteration 2, current learner xgboost
[flaml.automl.logger: 04-08 20:44:00] {2442} INFO - at -0.1s, estimator xgboost's best error=12.1446, best estimator rf's best error=7.9172
[flaml.automl.logger: 04-08 20:44:00] {2258} INFO - iteration 3, current learner extra_tree
[flaml.automl.logger: 04-08 20:44:00] {2442} INFO - at -0.0s, estimator extra_tree's best error=10.9591, best estimator rf's best error=7.9172
[flaml.automl.logger: 04-08 20:44:00] {2258} INFO - iteration 4, current learner xgb_limitdepth
[flaml.automl.logger: 04-08 20:44:01] {2442} INFO - at 0.5s, estimator xgb_limitdepth's best error=1.8870, best estimator xgb_limitdepth's best error=1.8870
[flaml.automl.logger: 04-08 20:44:01] {2258} INFO - iteration 5, current learner sgd
[flaml.automl.logger: 04-08 20:44:03] {2442} INFO - at 2.5s, estimator sgd's best error=14.5275, best estimator xgb_limitdepth's best error=1.8870
[flaml.automl.logger: 04-08 20:44:03] {2258} INFO - iteration 6, current learner lgbm
[flaml.automl.logger: 04-08 20:44:03] {2442} INFO - at 2.8s, estimator lgbm's best error=12.0178, best estimator xgb_limitdepth's best error=1.8870
[flaml.automl.logger: 04-08 20:44:03] {2258} INFO - iteration 7, current learner extra_tree
[flaml.automl.logger: 04-08 20:44:03] {2442} INFO - at 2.9s, estimator extra_tree's best error=8.3080, best estimator xgb_limitdepth's best error=1.8870
[flaml.automl.logger: 04-08 20:44:03] {2258} INFO - iteration 8, current learner lgbm
[flaml.automl.logger: 04-08 20:44:04] {2442} INFO - at 3.1s, estimator lgbm's best error=8.1328, best estimator xgb_limitdepth's best error=1.8870
```

▼ Code

```
1 print("RMSE using AutoML:", round(rmse, 2))
```

RMSE using AutoML: 2.19

Is this error high/low?

RMSE vs NRMSE

$$RMSE = \sqrt{\frac{1}{n} \sum (y_{true} - y_{pred})^2}$$

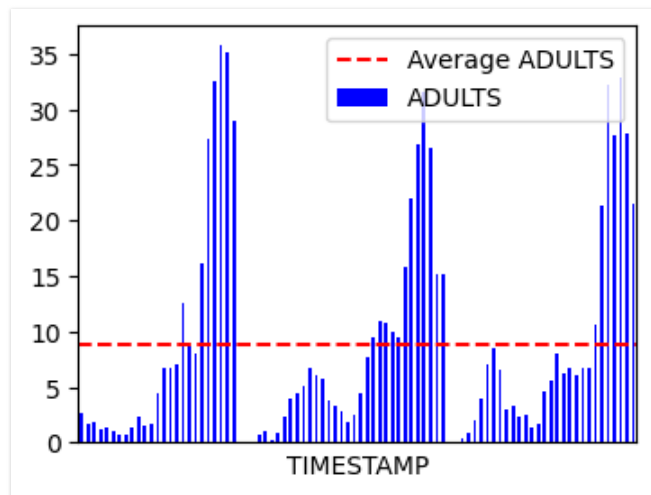
The squared differences have squared units, but the square root brings it back to the same unit as the target variable

$$NRMSE = \frac{RMSE}{avg(y_{true})}$$

▼ Code

```
1 import matplotlib.pyplot as plt
2 mean_df = df.groupby('TIMESTAMP')['ADULTS'].mean()
3 mean_df.plot(kind='bar', figsize=(4, 3), color='blue')
4 plt.xticks([]) # Hide the x-axis ticks
5 mean = mean_df.mean()
6 plt.axhline(y=mean, color='red', linestyle='--', label='Average ADULTS')
7 plt.legend()
8
9 print(f"RMSE {rmse:.3}, Mean {mean:.3}, NRMSE {rmse / mean:.2%}")
```

RMSE 2.19, Mean 8.92, NRMSE 24.52%



Let's try to better understand the data

▼ Code

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10949 entries, 0 to 10948
Data columns (total 16 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   GID                  10949 non-null  int64
 1   TIMESTAMP            10949 non-null  object
 2   CROP_ID              10949 non-null  int64
 3   ADULTS               10949 non-null  float64
 4   SMALL_INSTARS        10949 non-null  float64
 5   LARGE_INSTARS        10949 non-null  float64
 6   TEMPERATURE_AVG      10949 non-null  float64
 7   TEMPERATURE_MAX      10949 non-null  float64
 8   TEMPERATURE_MIN      10949 non-null  float64
 9   HUMIDITY_AVG         10949 non-null  float64
10   HUMIDITY_MAX         8180 non-null   float64
11   HUMIDITY_MIN         8180 non-null   float64
12   PRECIPITATIONS       10949 non-null  float64
13   WINDSPEED_AVG        8180 non-null   float64
14   WINDSPEED_MAX        8180 non-null   float64
15   TOTAL_CAPTURES       10949 non-null  float64
dtypes: float64(13), int64(2), object(1)
memory usage: 1.3+ MB
```

Let's try simpler models

▼ Code

```
1 from sklearn.linear_model import LinearRegression
2
3 X = df.drop(columns=['ADULTS']) # Drop the target variable 'ADULTS' from the feature set
4 X['TIMESTAMP'] = pd.to_datetime(X['TIMESTAMP']).astype(int) / 10**9 # Convert the 'TIMESTAMP' column to datetime and then to a Unix timestamp (seconds)
5 X.fillna(X.mean(), inplace=True) # Fill missing values with the mean of each column
6 y = df['ADULTS'] # Define the target variable
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Split the dataset into training and testing sets (80% train, 20% test)
8 log_reg = LinearRegression() # Initialize and train a Linear Regression model
9 log_reg.fit(X_train, y_train)
10 y_pred_log_reg = log_reg.predict(X_test) # Predict the target variable on the test set
11 # Calculate Root Mean Squared Error (RMSE)
12 rmse_log_reg = root_mean_squared_error(y_test, y_pred_log_reg)
13 print("RMSE using Linear Regression:", round(rmse_log_reg, 2))
```

RMSE using Linear Regression: 0.0

Why?

Let's take a look at the features. Do they ring a bell?

▼ Code

```
1 df.columns
```

```
Index(['GID', 'TIMESTAMP', 'CROP_ID', 'ADULTS', 'SMALL_INSTARS',  
      'LARGE_INSTARS', 'TEMPERATURE_AVG', 'TEMPERATURE_MAX',  
      'TEMPERATURE_MIN', 'HUMIDITY_AVG', 'HUMIDITY_MAX', 'HUMIDITY_MIN',  
      'PRECIPITATIONS', 'WINDSPEED_AVG', 'WINDSPEED_MAX', 'TOTAL_CAPTURES'],  
      dtype='object')
```

▼ Code

```
1 log_reg.coef_ # weights (coefficients) of the Linear Regression model
```

```
array([-1.36733869e-15,  9.57400348e-19,  3.29282180e-16, -1.00000000e+00,  
       -1.00000000e+00, -3.97477621e-15,  2.59706955e-15,  1.71515845e-15,  
        7.82904180e-16,  1.03568861e-15, -1.04128233e-15, -1.15471256e-16,  
       -1.29189124e-15, -3.68890616e-17,  1.00000000e+00])
```

LinearRegression correctly captures this pattern, AutoML does not

$ADULTS = - SMALL_INSTARS - LARGE_INSTARS + TOTAL_CAPTURES$

▼ Code

```
1 equation = " ".join(f"{' ' if weight < 0 else '+'}{round(weight, 2)}*{col}" for weight, col in zip(log_reg.coef_, X_train.columns) if abs(weight) > 0.01)  
2 print(f"ADULTS = {equation}")
```

```
ADULTS = -1.0*SMALL_INSTARS -1.0*LARGE_INSTARS +1.0*TOTAL_CAPTURES
```

Let's drop the **TOTAL_CAPTURES**

This was a trick, usually we do not have this type of derived features in the dataset

▼ Code

```
1 X = df.drop(columns=['ADULTS', 'TOTAL_CAPTURES'])
2 y = df['ADULTS']
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
4 automl = AutoML()
5 automl_settings = {
6     "time_budget": 30, # Time budget in seconds
7     "max_iter": 30, # Max number of iterations
8     "metric": 'rmse', # Evaluation metric
9     "task": 'regression', # Task type
10 }
11 automl.fit(X_train, y_train, **automl_settings)
12 y_pred = automl.predict(X_test)
13 rmse = root_mean_squared_error(y_test, y_pred)
```

```
[flaml.automl.logger: 04-08 20:44:12] {1728} INFO - task = regression
[flaml.automl.logger: 04-08 20:44:12] {1739} INFO - Evaluation method: holdout
[flaml.automl.logger: 04-08 20:44:12] {1838} INFO - Minimizing error metric: rmse
[flaml.automl.logger: 04-08 20:44:12] {1955} INFO - List of ML learners in AutoML Run: ['lgbm', 'rf', 'xgboost', 'extra_tree', 'xgb_limitdepth', 'sgd']
[flaml.automl.logger: 04-08 20:44:12] {2258} INFO - iteration 0, current learner lgbm
[flaml.automl.logger: 04-08 20:44:12] {2393} INFO - Estimated sufficient time budget=582s. Estimated necessary time budget=4s.
[flaml.automl.logger: 04-08 20:44:12] {2442} INFO - at 0.2s, estimator lgbm's best error=14.1248, best estimator lgbm's best error=14.1248
[flaml.automl.logger: 04-08 20:44:12] {2258} INFO - iteration 1, current learner lgbm
[flaml.automl.logger: 04-08 20:44:13] {2442} INFO - at 0.2s, estimator lgbm's best error=14.1248, best estimator lgbm's best error=14.1248
[flaml.automl.logger: 04-08 20:44:13] {2258} INFO - iteration 2, current learner sgd
[flaml.automl.logger: 04-08 20:44:14] {2442} INFO - at 2.1s, estimator sgd's best error=16.9466, best estimator lgbm's best error=14.1248
[flaml.automl.logger: 04-08 20:44:14] {2258} INFO - iteration 3, current learner lgbm
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.3s, estimator lgbm's best error=12.7827, best estimator lgbm's best error=12.7827
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 4, current learner xgboost
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.4s, estimator xgboost's best error=14.8380, best estimator lgbm's best error=12.7827
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 5, current learner lgbm
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.4s, estimator lgbm's best error=12.2781, best estimator lgbm's best error=12.2781
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 6, current learner lgbm
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.5s, estimator lgbm's best error=12.2781, best estimator lgbm's best error=12.2781
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 7, current learner lgbm
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.5s, estimator lgbm's best error=12.0187, best estimator lgbm's best error=12.0187
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 8, current learner lgbm
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.6s, estimator lgbm's best error=12.0187, best estimator lgbm's best error=12.0187
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 9, current learner lgbm
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.7s, estimator lgbm's best error=12.0187, best estimator lgbm's best error=12.0187
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 10, current learner lgbm
[flaml.automl.logger: 04-08 20:44:15] {2442} INFO - at 2.8s, estimator lgbm's best error=11.6593, best estimator lgbm's best error=11.6593
[flaml.automl.logger: 04-08 20:44:15] {2258} INFO - iteration 11, current learner xgboost
```

▼ Code

```
1 print("RMSE using AutoML:", round(rmse, 2))
```

RMSE using AutoML: 11.13

▼ Code

```
1 automl.best_estimator
```

'lgbm'

▼ Code

```
1 automl.best_config
```

```
{'n_estimators': 99,  
 'num_leaves': 15,  
 'min_child_samples': 3,  
 'learning_rate': np.float64(0.17402065726724145),  
 'log_max_bin': 8,  
 'colsample_bytree': np.float64(0.6649148062238498),  
 'reg_alpha': 0.0009765625,  
 'reg_lambda': np.float64(0.006761362450996487)}
```

Can we do better?

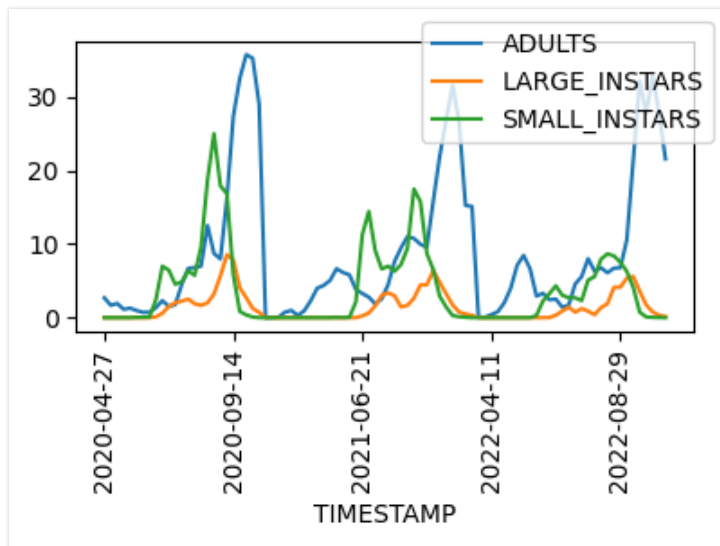
From the literature, we know that the spreading of bugs depends on:

1. Cumulative degree days: $\sum_{d \in [May-Sep]} \max(0, T_d^{avg} - 12.2)$ ([Forresi et al. 2024](#))
2. Time. How?

Captures over time

▼ Code

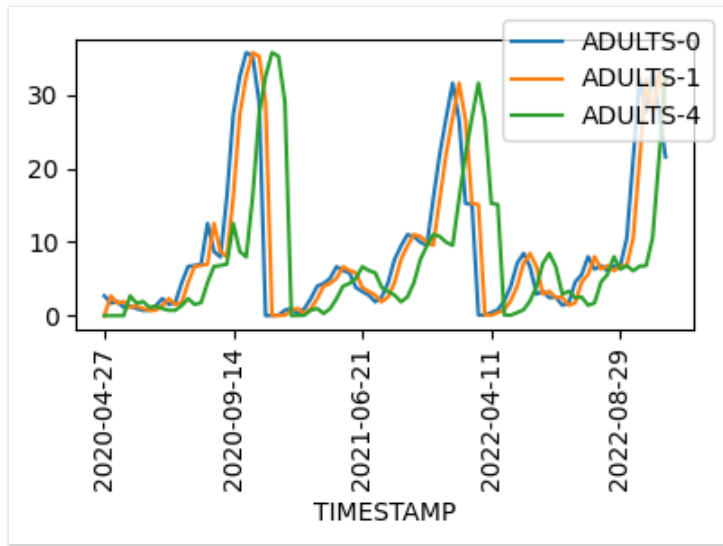
```
1 fig, ax = plt.subplots(1, 1, figsize=(4, 3))
2
3 for c in ["ADULTS", "LARGE_INSTARS", "SMALL_INSTARS"]:
4     df.groupby("TIMESTAMP")[c].mean().plot(x="TIMESTAMP", y=c, ax=ax)
5
6 plt.xticks(rotation=90)
7 fig.legend()
8 fig.tight_layout()
```



Predicting Adults with past (previous) Adults

▼ Code

```
1 fig, ax = plt.subplots(1, 1, figsize=(4, 3))
2
3 for c in [0, 1, 4]:
4     df.groupby('TIMESTAMP')['ADULTS'].mean().shift(c).fillna(0).plot(x="ADULTS", y=c, ax=ax, label=f"ADULTS-{c}")
5
6 plt.xticks(rotation=90)
7 fig.legend()
8 fig.tight_layout()
```



Preprocessing

▼ Code

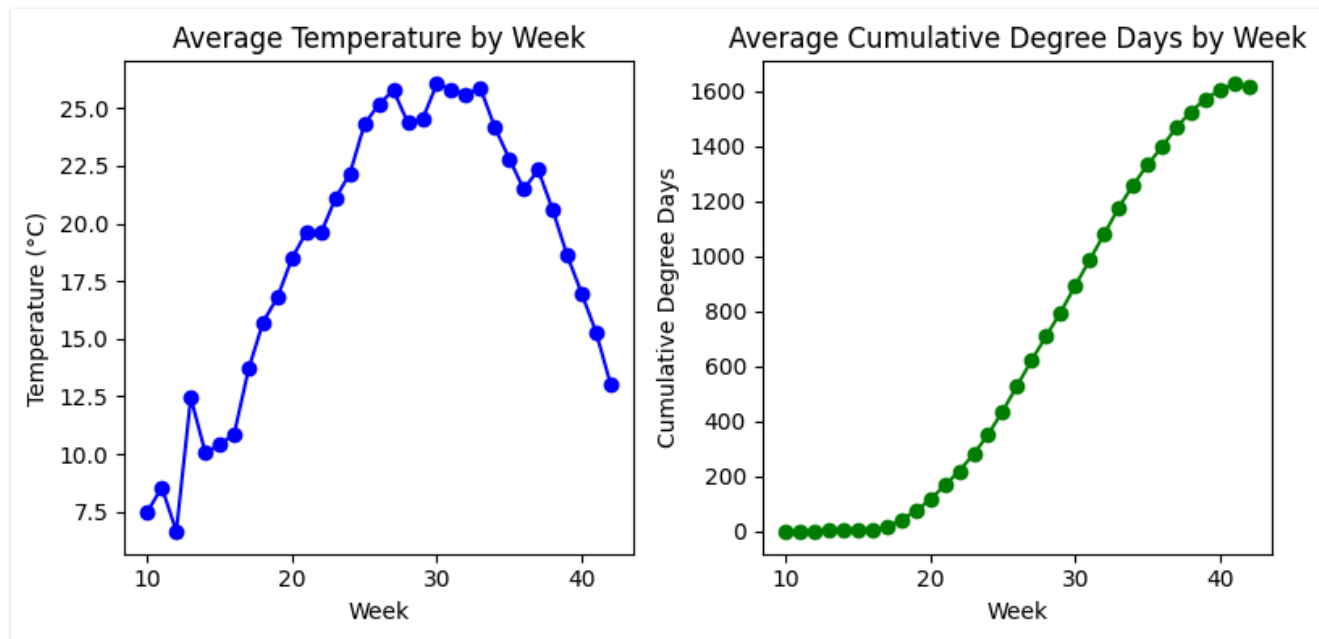
```
1 df = pd.read_csv("../datasets/cimice/captures-raw.csv", sep=',') # Load the dataset from a CSV file
2
3 df['TIMESTAMP'] = pd.to_datetime(df['TIMESTAMP']) # Convert 'TIMESTAMP' to datetime format for time-based analysis
4 df.sort_values(by=['GID', 'TIMESTAMP'], inplace=True) # Sort dataset by 'GID' (Group ID) and 'TIMESTAMP' to maintain chronological order
5
6 df['WEEK'] = df['TIMESTAMP'].dt.isocalendar().week # Extract the week number from the timestamp for seasonal analysis
7
8 df['DEGREE_DAYS'] = df['TEMPERATURE_AVG'].apply(lambda x: max(0, x - 12.2)) # Calculate degree days: only consider temperatures above 12.2°C
9 df['DAYS_DIFF'] = df.groupby('GID')['TIMESTAMP'].diff().dt.days.fillna(14) # Compute the time difference (in days) between consecutive observations per 'GID'
10 df['CUM_DEGREE_DAYS'] = df.groupby('GID').apply(lambda group: (group['DEGREE_DAYS'] * group['DAYS_DIFF']).cumsum()).reset_index(level=0, drop=True) # Compute cumulative degree days for each 'GID'
11
12 df['ADULTS-1'] = df.groupby('GID')['ADULTS'].shift(1).fillna(0) # Create a lag feature: previous 'ADULTS' count for each 'GID', fill missing values with 0
13
14 df['TIMESTAMP'] = df['TIMESTAMP'].astype(int) / 10**9 # Convert 'TIMESTAMP' to Unix timestamp (seconds) for numerical processing
15
16 df.fillna(df.groupby('WEEK').transform('mean'), inplace=True) # Fill missing values using the mean of each column, grouped by 'WEEK'
17
18 df.drop(columns=['TOTAL_CAPTURES'], inplace=True)
19 X = df.drop(columns=['ADULTS'])
20 y = df['ADULTS']
21 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
22
23 df.head()
```

	GID	TIMESTAMP	CROP_ID	ADULTS	SMALL_INSTARS	LARGE_INSTARS	TEMPERATURE_AVG	TEMPERATURE_MAX	TEMPERATURE_MIN	HUMIDITY_AVG	HUMIDITY_MAX	HUMIDITY_MIN	PRECIPITATIONS	WIND_SPEED
7	24	1.587946e+09	47	2.0	0.0	0.0	16.871423	23.171424	9.642855	61.142857	84.571429	41.285714	3.199999	2.11
43	24	1.588550e+09	47	1.0	0.0	0.0	16.700002	22.900001	9.314286	62.714286	86.857143	37.714286	0.000000	2.00
194	24	1.589155e+09	47	1.0	0.0	0.0	18.485712	24.899997	12.028571	65.857143	87.571429	42.714286	3.199997	1.88
358	24	1.589760e+09	47	2.0	0.0	0.0	19.362497	24.737505	13.624998	67.875000	89.375000	46.250000	2.000004	1.74
453	24	1.590365e+09	47	0.0	0.0	0.0	19.699996	25.883335	11.683333	55.333333	84.500000	32.500000	2.600000	2.00

Average temperature vs cumulative degree days

▼ Code

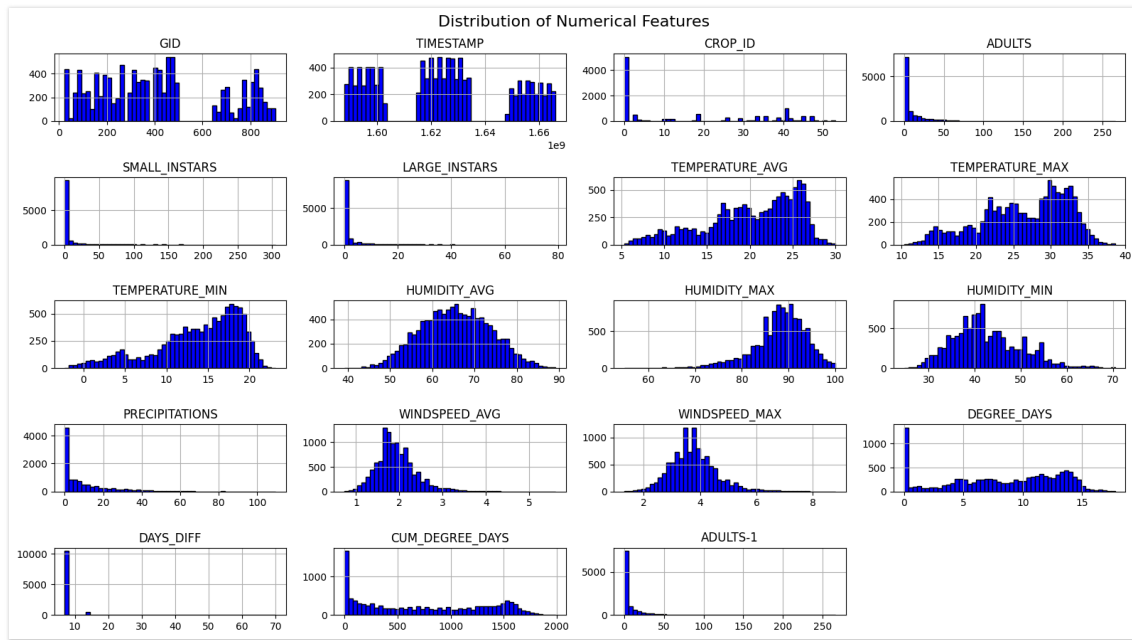
```
1 import matplotlib.pyplot as plt
2 # Calculate weekly averages for temperature and cumulative degree days
3 weekly_avg_temp = df.groupby('WEEK')['TEMPERATURE_AVG'].mean()
4 weekly_avg_cum_degree_days = df.groupby('WEEK')['CUM_DEGREE_DAYS'].mean()
5 fig, axes = plt.subplots(1, 2, figsize=(8, 4)) # Create subplots
6 axes[0].plot(weekly_avg_temp.index, weekly_avg_temp.values, marker='o', color='blue') # Plot average temperature by week
7 axes[0].set_title('Average Temperature by Week')
8 axes[0].set_xlabel('Week')
9 axes[0].set_ylabel('Temperature (°C)')
10 axes[1].plot(weekly_avg_cum_degree_days.index, weekly_avg_cum_degree_days.values, marker='o', color='green') # Plot average cumulative degree days by week
11 axes[1].set_title('Average Cumulative Degree Days by Week')
12 axes[1].set_xlabel('Week')
13 axes[1].set_ylabel('Cumulative Degree Days')
14 plt.tight_layout()
```



Data understanding

▼ Code

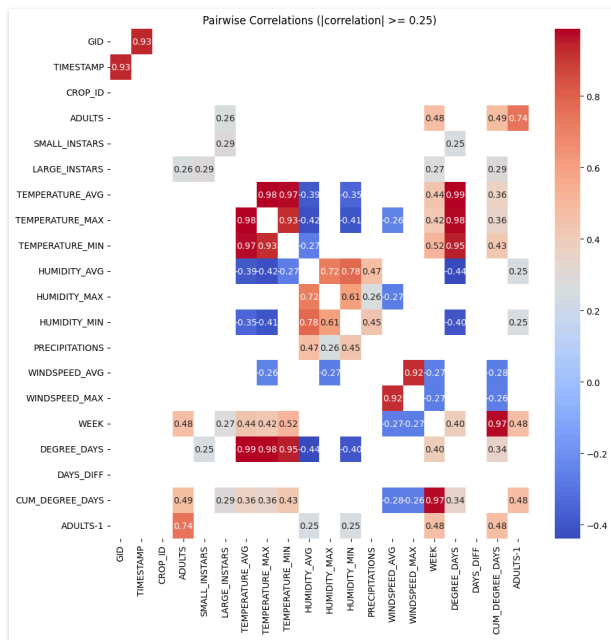
```
1 numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns # Select numerical columns
2 df[numerical_columns].hist(bins=50, figsize=(16, 9), color='blue', edgecolor='black') # Plot histograms for numerical columns
3 plt.suptitle('Distribution of Numerical Features', fontsize=16)
4 plt.tight_layout()
```



Data understanding

▼ Code

```
1 import seaborn as sns
2 import numpy as np
3 correlation_matrix = df.corr() # Compute the correlation matrix
4 filtered_correlation_matrix = correlation_matrix[correlation_matrix.abs() >= 0.25] # Mask correlations with absolute values below 0.25
5 filtered_correlation_matrix[np.eye(filtered_correlation_matrix.shape[0], dtype=bool)] = np.nan # Mask the diagonal
6 plt.figure(figsize=(10, 10))
7 sns.heatmap(filtered_correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
8 plt.title("Pairwise Correlations (|correlation| >= 0.25)")
9 plt.tight_layout()
```

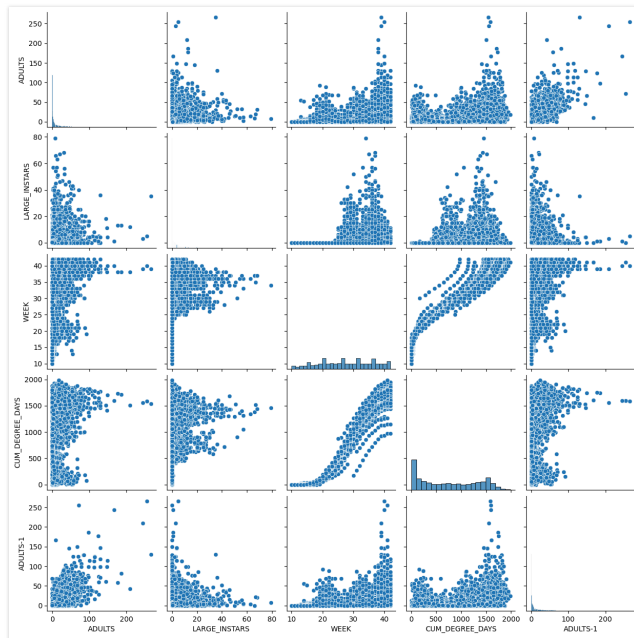


Plotting the correlations

▼ Code

```
1 plt.figure(figsize=(10, 10))
2 sns.pairplot(df[["ADULTS", "LARGE_INSTARS", "WEEK", "CUM_DEGREE_DAYS", "ADULTS-1"]])
```

<Figure size 1000x1000 with 0 Axes>



Let's apply simple models

▼ Code

```
1 from sklearn.linear_model import LinearRegression
2 log_reg = LinearRegression()
3 log_reg.fit(X_train, y_train)
4 y_pred_log_reg = log_reg.predict(X_test)
5 rmse_log_reg = root_mean_squared_error(y_test, y_pred_log_reg)
6 print("RMSE using Logistic Regression:", round(rmse_log_reg, 2))
```

RMSE using Logistic Regression: 9.88

▼ Code

```
1 from sklearn.ensemble import RandomForestRegressor
2 rf_regressor = RandomForestRegressor(random_state=42)
3 rf_regressor.fit(X_train, y_train)
4 y_pred_rf = rf_regressor.predict(X_test)
5 rmse_rf = root_mean_squared_error(y_test, y_pred_rf)
6 print("RMSE using Random Forest Regressor:", round(rmse_rf, 2))
```

RMSE using Random Forest Regressor: 9.57

Let's do hyper-parameter tuning on RF

▼ Code

```
1 from sklearn.model_selection import RandomizedSearchCV
2 rf = RandomForestRegressor(random_state=42) # Define the base model
3 param_dist = { # Define the hyperparameter grid
4     'n_estimators': [50, 100, 200, 300, 400],
5     'max_depth': [10, 20, 30],
6     'min_samples_split': [2, 5, 10],
7     'min_samples_leaf': [1, 2, 4],
8     'max_features': ['sqrt', 'log2']
9 }
10 random_search = RandomizedSearchCV(estimator=rf, param_distributions=param_dist, n_iter=20, cv=5, scoring='neg_root_mean_squared_error', verbose=2, n_jobs=-1, random_state=42) # Use Randomized Search
11 random_search.fit(X_train, y_train) # Best parameters from Randomized Search
12 best_model = random_search.best_estimator_
13 y_pred_rf = best_model.predict(X_test) # Make predictions
14 rmse_rf = root_mean_squared_error(y_test, y_pred_rf) # Compute RMSE
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

```
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, min_samples_split=2, n_estimators=50; total time= 1.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, min_samples_split=2, n_estimators=50; total time= 1.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, min_samples_split=2, n_estimators=50; total time= 1.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, min_samples_split=2, n_estimators=50; total time= 2.6s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, min_samples_split=2, n_estimators=50; total time= 2.7s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=100; total time= 4.7s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=100; total time= 4.9s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=100; total time= 4.9s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=100; total time= 7.4s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=100; total time= 7.7s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=4, min_samples_split=2, n_estimators=400; total time= 13.7s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=4, min_samples_split=5, n_estimators=200; total time= 9.2s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=4, min_samples_split=2, n_estimators=400; total time= 14.1s
[CV] END max_depth=30, max_features=sqrt, min_samples_leaf=2, min_samples_split=2, n_estimators=100; total time= 6.0s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=4, min_samples_split=5, n_estimators=200; total time= 9.5s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=4, min_samples_split=5, n_estimators=200; total time= 9.5s
[CV] END max_depth=30, max_features=sqrt, min_samples_leaf=2, min_samples_split=2, n_estimators=100; total time= 5.9s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=4, min_samples_split=2, n_estimators=400; total time= 13.7s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=4, min_samples_split=5, n_estimators=200; total time= 11.8s
[CV] END max_depth=20, max_features=sqrt, min_samples_leaf=4, min_samples_split=5, n_estimators=200; total time= 11.7s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=4, min_samples_split=2, n_estimators=400; total time= 13.9s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=4, min_samples_split=2, n_estimators=400; total time= 14.5s
[CV] END max_depth=30, max_features=sqrt, min_samples_leaf=2, min_samples_split=2, n_estimators=100; total time= 5.3s
[CV] END max_depth=30, max_features=sqrt, min_samples_leaf=2, min_samples_split=2, n_estimators=100; total time= 5.3s
[CV] END max_depth=30, max_features=sqrt, min_samples_leaf=2, min_samples_split=2, n_estimators=100; total time= 6.0s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=1, min_samples_split=2, n_estimators=50; total time= 1.0s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=1, min_samples_split=2, n_estimators=50; total time= 1.0s
```

▼ Code

```
1 random_search.best_params_
```

```
{'n_estimators': 200,  
 'min_samples_split': 2,  
 'min_samples_leaf': 1,  
 'max_features': 'log2',  
 'max_depth': 30}
```

▼ Code

```
1 random_search.best_estimator_
```

▼ RandomForestRegressor



```
RandomForestRegressor(max_depth=30, max_features='log2', n_estimators=200,  
                      random_state=42)
```

▼ Code

```
1 print("Optimized RMSE using Random Forest Regressor:", round(rmse_rf, 2))
```

Optimized RMSE using Random Forest Regressor: 9.49

Let's try more complex models

▼ Code

```
1 import xgboost as xgb
2 xgb_regressor = xgb.XGBRegressor(objective='reg:squarederror', random_state=42) # Define base XGBoost model
3 param_dist = { # Define hyperparameter grid
4     'n_estimators': [100, 200, 300, 500],
5     'learning_rate': [0.01, 0.05, 0.1, 0.2],
6     'max_depth': [3, 5, 7, 10],
7     'min_child_weight': [1, 3, 5, 7],
8     'subsample': [0.6, 0.8, 1.0],
9     'colsample_bytree': [0.6, 0.8, 1.0],
10    'gamma': [0, 0.1, 0.2, 0.5]
11 }
12 # Perform Randomized Search for broad tuning
13 random_search = RandomizedSearchCV(
14     estimator=xgb_regressor, param_distributions=param_dist,
15     n_iter=30, cv=5, scoring='neg_root_mean_squared_error',
16     verbose=2, n_jobs=-1, random_state=42
17 )
18 random_search.fit(X_train, y_train) # Get best parameters from Randomized Search
19 best_xgb = random_search.best_estimator_
20 y_pred_xgb = best_xgb.predict(X_test) # Make predictions
21 rmse_xgb = root_mean_squared_error(y_test, y_pred_xgb) # Compute RMSE
```

Fitting 5 folds for each of 30 candidates, totalling 150 fits

```
[CV] END colsample_bytree=0.8, gamma=0.5, learning_rate=0.01, max_depth=3, min_child_weight=3, n_estimators=100, subsample=1.0; total time= 0.6s
[CV] END colsample_bytree=0.8, gamma=0.5, learning_rate=0.01, max_depth=3, min_child_weight=3, n_estimators=100, subsample=1.0; total time= 0.7s
[CV] END colsample_bytree=0.8, gamma=0.5, learning_rate=0.01, max_depth=3, min_child_weight=3, n_estimators=100, subsample=1.0; total time= 0.6s
[CV] END colsample_bytree=0.8, gamma=0.5, learning_rate=0.01, max_depth=3, min_child_weight=3, n_estimators=100, subsample=1.0; total time= 0.6s
[CV] END colsample_bytree=0.8, gamma=0.5, learning_rate=0.01, max_depth=3, min_child_weight=3, n_estimators=100, subsample=1.0; total time= 0.6s
[CV] END colsample_bytree=0.6, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=7, n_estimators=300, subsample=1.0; total time= 2.3s
[CV] END colsample_bytree=0.6, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=7, n_estimators=300, subsample=1.0; total time= 2.3s
[CV] END colsample_bytree=0.6, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=7, n_estimators=300, subsample=1.0; total time= 2.7s
[CV] END colsample_bytree=0.8, gamma=0.2, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=300, subsample=0.8; total time= 1.6s
[CV] END colsample_bytree=0.8, gamma=0.2, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=300, subsample=0.8; total time= 1.4s
[CV] END colsample_bytree=0.8, gamma=0.2, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=300, subsample=0.8; total time= 1.4s
[CV] END colsample_bytree=0.8, gamma=0.2, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=300, subsample=0.8; total time= 1.5s
[CV] END colsample_bytree=0.8, gamma=0.2, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=300, subsample=0.8; total time= 1.6s
[CV] END colsample_bytree=0.6, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=7, n_estimators=300, subsample=1.0; total time= 4.9s
[CV] END colsample_bytree=0.6, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=7, n_estimators=300, subsample=1.0; total time= 5.0s
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.01, max_depth=7, min_child_weight=5, n_estimators=100, subsample=0.8; total time= 5.4s
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.01, max_depth=7, min_child_weight=5, n_estimators=100, subsample=0.8; total time= 5.7s
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.01, max_depth=7, min_child_weight=5, n_estimators=100, subsample=0.8; total time= 2.5s
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.01, max_depth=7, min_child_weight=5, n_estimators=100, subsample=0.8; total time= 2.3s
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.01, max_depth=7, min_child_weight=5, n_estimators=100, subsample=0.8; total time= 2.3s
[CV] END colsample_bytree=0.6, gamma=0, learning_rate=0.1, max_depth=5, min_child_weight=5, n_estimators=500, subsample=0.8; total time= 2.5s
[CV] END colsample_bytree=0.6, gamma=0, learning_rate=0.1, max_depth=5, min_child_weight=5, n_estimators=500, subsample=0.8; total time= 3.8s
```

▼ Code

```
1 random_search.best_params_
```

```
{'subsample': 0.6,  
 'n_estimators': 500,  
 'min_child_weight': 5,  
 'max_depth': 7,  
 'learning_rate': 0.01,  
 'gamma': 0.5,  
 'colsample_bytree': 0.6}
```

▼ Code

```
1 random_search.best_estimator_
```

▼

XGBRegressor

i

```
XGBRegressor(base_score=None, booster=None, callbacks=None,  
             colsample_bylevel=None, colsample_bynode=None,  
             colsample_bytree=0.6, device=None, early_stopping_rounds=None,  
             enable_categorical=False, eval_metric=None, feature_types=None,  
             gamma=0.5, grow_policy=None, importance_type=None,  
             interaction_constraints=None, learning_rate=0.01, max_bin=None,  
             max_cat_threshold=None, max_cat_to_onehot=None,  
             max_delta_step=None, max_depth=7, max_leaves=None,  
             min_child_weight=5, missing=nan, monotone_constraints=None,  
             multi_strategy=None, n_estimators=500, n_jobs=None,  
             num_parallel_tree=None, random_state=42, ...)
```

▼ Code

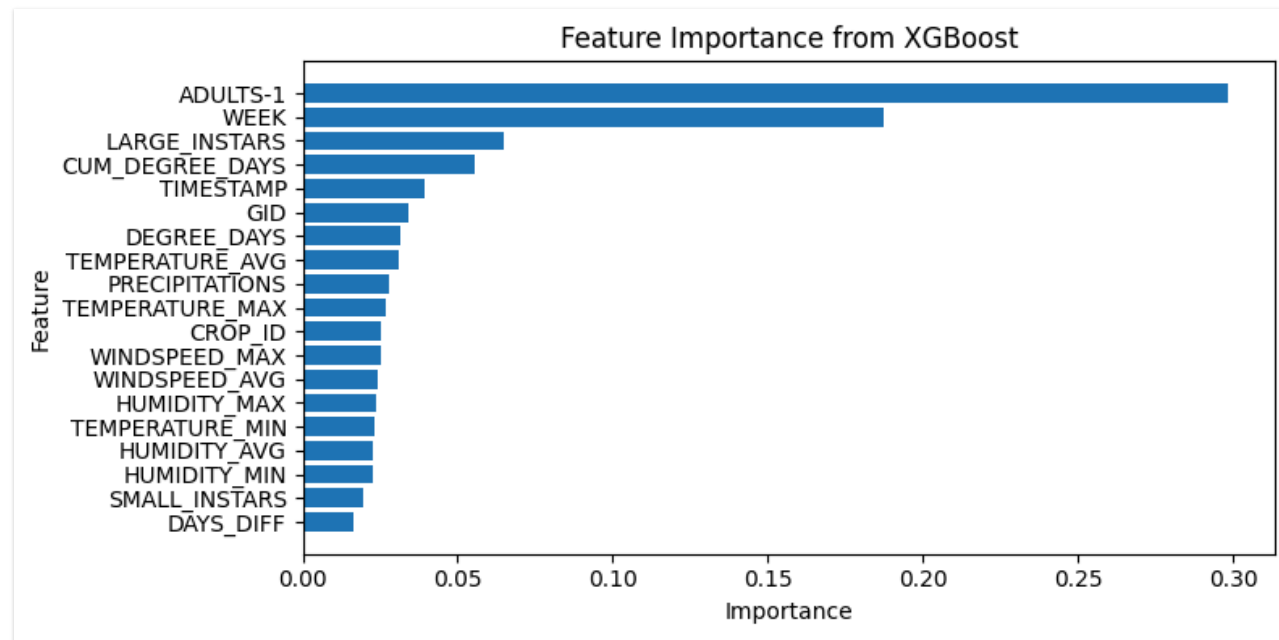
```
1 print("Optimized RMSE using XGBoost:", round(rmse_xgb, 2))
```

Optimized RMSE using XGBoost: 9.48

Plotting feature importance

▼ Code

```
1 feature_importance = best_xgb.feature_importances_ # Get feature importance
2 feature_importance_df = pd.DataFrame({ 'Feature': X_train.columns, 'Importance': feature_importance }).sort_values(by='Importance', ascending=False)
3 plt.figure(figsize=(8, 4))
4 plt.barh(feature_importance_df['Feature'], feature_importance_df['Importance'])
5 plt.xlabel('Importance')
6 plt.ylabel('Feature')
7 plt.title('Feature Importance from XGBoost')
8 plt.gca().invert_yaxis()
9 plt.tight_layout()
```



Let's retry AutoML

▼ Code

```
1 automl = AutoML()
2 automl_settings = {
3     "time_budget": 30, # Time budget in seconds
4     "max_iter": 30, # Max number of iterations
5     "metric": 'rmse', # Evaluation metric
6     # "estimator_list": ["xgboost"],
7     "task": 'regression', # Task type
8 }
9 automl.fit(X_train, y_train, **automl_settings)
10 y_pred = automl.predict(X_test)
11 rmse = root_mean_squared_error(y_test, y_pred)
```

```
[flaml.automl.logger: 04-08 20:50:07] {1728} INFO - task = regression
[flaml.automl.logger: 04-08 20:50:07] {1739} INFO - Evaluation method: holdout
[flaml.automl.logger: 04-08 20:50:07] {1838} INFO - Minimizing error metric: rmse
[flaml.automl.logger: 04-08 20:50:07] {1955} INFO - List of ML learners in AutoML Run: ['lgbm', 'rf', 'xgboost', 'extra_tree', 'xgb_limitdepth', 'sgd']
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 0, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2393} INFO - Estimated sufficient time budget=561s. Estimated necessary time budget=4s.
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.2s, estimator lgbm's best error=13.7019, best estimator lgbm's best error=13.7019
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 1, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.3s, estimator lgbm's best error=13.7019, best estimator lgbm's best error=13.7019
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 2, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.3s, estimator lgbm's best error=11.1891, best estimator lgbm's best error=11.1891
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 3, current learner sgd
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.4s, estimator sgd's best error=18.1963, best estimator lgbm's best error=11.1891
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 4, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.4s, estimator lgbm's best error=10.0279, best estimator lgbm's best error=10.0279
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 5, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.5s, estimator lgbm's best error=10.0279, best estimator lgbm's best error=10.0279
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 6, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.5s, estimator lgbm's best error=9.9213, best estimator lgbm's best error=9.9213
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 7, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.6s, estimator lgbm's best error=9.9213, best estimator lgbm's best error=9.9213
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 8, current learner lgbm
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.6s, estimator lgbm's best error=9.9213, best estimator lgbm's best error=9.9213
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 9, current learner sgd
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.7s, estimator sgd's best error=18.1868, best estimator lgbm's best error=9.9213
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 10, current learner xgboost
[flaml.automl.logger: 04-08 20:50:07] {2442} INFO - at 0.8s, estimator xgboost's best error=13.7583, best estimator lgbm's best error=9.9213
[flaml.automl.logger: 04-08 20:50:07] {2258} INFO - iteration 11, current learner lgbm
```

▼ Code

```
1 automl.best_estimator
```

```
'lgbm'
```

▼ Code

```
1 automl.best_config
```

```
{'n_estimators': 34,  
 'num_leaves': 4,  
 'min_child_samples': 4,  
 'learning_rate': np.float64(0.41929025492645006),  
 'log_max_bin': 8,  
 'colsample_bytree': np.float64(0.7610534336273627),  
 'reg_alpha': 0.0009765625,  
 'reg_lambda': np.float64(0.009280655005879927)}
```

▼ Code

```
1 print("RMSE using AutoML:", round(rmse, 2))
```

```
RMSE using AutoML: 9.77
```

Questions

- Can this model generalize to any country? E.g., to Australia

References

- Bie, Tijl De, Luc De Raedt, José Hernández-Orallo, Holger H. Hoos, Padhraic Smyth, and Christopher K. I. Williams. 2022. “Automating Data Science.” *Commun. ACM* 65 (3): 76–87.
- Forresi, Chiara, Enrico Gallinucci, Matteo Golfarelli, Lara Maistrello, Michele Preti, and Giacomo Vaccari. 2024. “A Data Platform for Real-Time Monitoring and Analysis of the Brown Marmorated Stink Bug in Northern Italy.” *Ecol. Informatics* 82: 102713. <https://doi.org/10.1016/J.ECOINF.2024.102713>.
- Waldo, Jim, and Soline Boussard. 2025. “GPTs and Hallucination.” *Commun. ACM* 68 (1): 40–45.
- Welsh, Matt. 2023. “The End of Programming.” *Commun. ACM* 66 (1): 34–35.